

AI4GAMMA

Corso di Formazione su Claude

Manuale Utente — Quarta Lezione

Data sessione: 10 aprile 2026

Docente: Stefano Zaghi

Partecipanti: Pierpaolo Laveni, Christian Frassetto, Walter Leggendari, Paolo Bergamaschi

Piattaforma: Microsoft Teams | Durata: ~2,5 ore

Sommario

Introduzione	4
1 — Recap della lezione precedente.....	5
1.1 Il file CLAUDE.md come memoria statica	5
1.2 Modalità di lavoro di Claude Code	5
1.3 Effort e gestione dei token	6
1.4 Accesso a Claude Code	6
2 — Plugin di Claude e Claude Code.....	7
2.1 Cosa sono i plugin	7
2.2 Dove si trovano e come si installano i plugin	8
2.3 Gestione dei plugin nell'organizzazione	8
2.4 Plugin fondamentali per la metodologia SDD.....	9
3 — Spec-Driven Development.....	10
3.1 Definizione.....	10
3.2 SDD vs Vibe Coding	10
3.3 I 5 principi fondamentali dello SDD	11
Principio 1 — Separazione del cosa dal come	11
Principio 2 — Granularità controllata.....	11
Principio 3 — Review iterativa con cicli di feedback	11
Principio 4 — Controllo umano sulle decisioni critiche	11
Principio 5 — La specifica come documentazione di progetto.....	11
3.4 Il ciclo di sviluppo SDD	12
3.5 I due tipi di specifiche	12
3.6 I 7 componenti di una specifica efficace	13
4 — Workflow SDD Tradizionale (Planning Mode).....	14
4.1 Le 6 fasi del Workflow Tradizionale	14
4.2 Caso pratico: EV-002 — Configurable Working Hours.....	15
5 — Workflow SDD Avanzato (Brainstorming & Specification).....	16
5.1 L'idea di fondo	16
5.2 Le fasi del Workflow Avanzato.....	16
5.3 Il plugin SuperPowers e il comando /brainstorming	17
5.4 Caso pratico: EV-003 — Modified Gantt Date Calculation	17
5.5 Confronto tra Workflow Tradizionale e Avanzato	18
5.6 Quando usare quale workflow	18
6 — Dimostrazione pratica: App Shopping List	19

6.1 Avvio del brainstorming	19
6.2 Sessione di Q&A esplorativo	19
6.3 Generazione della specifica e del piano	20
6.4 Risultato del prototipo	20
7 — Esercizi assegnati.....	21
Esercizio 1 — Scrittura manuale delle specifiche	21
Esercizio 2 — Configurazione dell'ambiente Claude Code	21
Esercizio 3 — Scrittura delle specifiche con Brainstorming.....	21
Esercizio 4 — Implementazione del prototipo	21
Riepilogo e Concetti Chiave	22
Plugin di Claude e Claude Code.....	22
Metodologia Spec-Driven Development.....	22
Workflow Tradizionale vs Avanzato	22

Introduzione

La sessione, della durata di circa 2 ore e 30 minuti, ha introdotto la metodologia Spec-Driven Development (SDD), illustrato il sistema dei plugin di Claude Code, e mostrato — attraverso dimostrazioni pratiche — come applicare un approccio strutturato e controllato allo sviluppo assistito dall'AI.

Il filo conduttore dell'intera lezione è sintetizzabile in un principio cardine: la qualità del risultato che ottenete dall'AI è direttamente proporzionale alla qualità della specifica che le fornite.

Tutto ciò che viene trattato — plugin, workflow, CLAUDE.md, prompt template — converge verso questo obiettivo: rendere la collaborazione uomo-AI prevedibile, ripetibile e di alta qualità.

Obiettivi di apprendimento della lezione
Comprendere cos'è la metodologia Spec-Driven Development
Conoscere il sistema dei plugin di Claude Code e sapere come installarli
Applicare il Workflow SDD Tradizionale (Planning Mode) con le sue 6 fasi
Applicare il Workflow SDD Avanzato (Brainstorming & Specification) con il plugin SuperPowers
Comprendere il ruolo del file CLAUDE.md

1 — Recap della lezione precedente

La lezione si è aperta con un riepilogo dettagliato della sessione precedente, dedicata a Claude Code. I punti chiave già affrontati e qui richiamati come base di partenza sono i seguenti.

1.1 Il file **CLAUDE.md** come memoria statica

Claude Code richiede un file di configurazione chiamato **CLAUDE.md** (estensione Markdown) che viene caricato in memoria all'avvio di ogni sessione. Questo file rappresenta la memoria statica del progetto: contiene il ruolo dell'agente, le sue competenze, le istruzioni chiave e l'obiettivo da raggiungere.

Il file può essere posizionato direttamente nella root della cartella di progetto oppure, preferibilmente, all'interno di una sottocartella nascosta denominata `“.claude”`, che Claude stesso può creare per raccogliere tutti i file di configurazione e di riferimento.

Best Practice

*Creare manualmente la cartella `.claude` se Claude non la crea in autonomia, e posizionarvi il file **CLAUDE.md**. Questo mantiene il progetto ordinato e permette a Claude di trovare la configurazione in modo affidabile.*

1.2 Modalità di lavoro di Claude Code

Claude Code offre quattro modalità operative selezionabili dal menu a tendina nell'interfaccia:

Modalità	Descrizione e utilizzo raccomandato
Ask Permissions (standard)	Claude chiede il permesso prima di eseguire qualsiasi azione. La modalità raccomandata per mantenere il controllo sul processo.
Auto Accept Edits	Claude esegue tutte le modifiche senza chiedere conferma. Molto rapida ma potenzialmente rischiosa — da usare con cautela su progetti consolidati.
Planning Mode	Forza Claude a esplorare la codebase e produrre un piano di sviluppo dettagliato prima di scrivere codice. Ideale per iniziare un nuovo task.
Auto Mode	Claude decide autonomamente quando chiedere o non chiedere permessi, ottimizzando per disturbare il meno possibile il flusso di lavoro. Pensato per attività parallele.

Flusso raccomandato

*Iniziare sempre in **Planning Mode** per produrre il piano di sviluppo. Una volta approvato il piano, passare alla modalità **Ask Permissions (standard)** per l'esecuzione controllata.*

1.3 Effort e gestione dei token

Dal menu di selezione del modello è disponibile l'opzione Effort, che controlla l'intensità del ragionamento interno di Claude (thinking mode):

- Low: Claude ragiona poco, consuma pochi token, risponde velocemente. Adatto per task semplici e ripetitivi.
- Medium (raccomandato): bilanciamento ottimale tra qualità del ragionamento e consumo di token. La context window si riempie lentamente, mantenendo alta la qualità delle risposte anche nelle sessioni lunghe.
- High: Claude ragiona estensivamente, rivede le proprie scelte, ma consuma molti token e riempie rapidamente la context window. Non sempre porta a risultati migliori.

1.4 Accesso a Claude Code

Claude Code è accessibile indifferentemente dall'interfaccia grafica di Claude Desktop oppure da terminale. Da terminale (Visual Studio Code, CMD, PowerShell) è sufficiente digitare il comando "claude" per avviare una sessione nella cartella corrente, oppure navigare prima nella cartella desiderata (es. `cd D:\test`) e poi digitare `claude`. In entrambi i casi, il file `CLAUDE.md` presente nella cartella viene caricato automaticamente in memoria.

2 — Plugin di Claude e Claude Code

Prima di affrontare la metodologia SDD, è fondamentale comprendere il sistema dei plugin, che rappresenta il meccanismo attraverso il quale Claude Code estende le proprie capacità e che abilita il Workflow SDD Avanzato illustrato nel capitolo successivo.

2.1 Cosa sono i plugin

I plugin sono pacchetti di funzionalità che estendono le capacità native di Claude. Ogni plugin è composto da uno o più dei seguenti componenti:

Componente	Descrizione
Skills	Competenze specializzate: capacità predefinite che Claude può applicare su richiesta (es. tradurre un documento, generare un piano di sviluppo secondo uno schema preciso, revisionare il CLAUDE.md).
Connectors / MCP	Connessioni verso sistemi e basi dati esterne tramite il protocollo MCP (Model Context Protocol). Permettono a Claude di interrogare database, API, file system remoti.
Sub-agents	Agenti specializzati configurabili per svolgere compiti specifici all'interno di Claude Code.
Hooks	Automazioni basate su eventi (es. all'invio di un prompt, alla ricezione di una risposta). Da usare con cautela — possono generare comportamenti inattesi.

 Nota

In questo corso verranno approfonditi Skills e Connectors MCP. Sub-agents e Hooks sono argomenti avanzati che esulano dagli obiettivi attuali.

2.2 Dove si trovano e come si installano i plugin

I plugin per Claude Code si installano da riga di comando — non dall'interfaccia grafica. Il portale di riferimento è accessibile tramite browser e contiene la lista di tutti i plugin ufficiali disponibili, filtrabili per compatibilità con Claude Code.

La procedura di installazione è la seguente:

1. Aprire il portale dei plugin nel browser.
2. Trovare il plugin desiderato e copiare il comando di installazione tramite il pulsante apposito.
3. Aprire un terminale (CMD, PowerShell o il terminale integrato in VS Code) — non è necessario trovarsi all'interno di una sessione Claude Code.
4. Incollare ed eseguire il comando, che ha la forma: `claude plugin install <nome-plugin>`
5. Attendere la conferma di installazione.
6. Chiudere e riavviare completamente Claude Desktop (File > Quit, poi riapertura), affinché i nuovi plugin vengano caricati in memoria.

Attenzione

Il riavvio di Claude Desktop dopo l'installazione di un plugin è obbligatorio. Senza di esso, i nuovi plugin non saranno disponibili nella sessione corrente.

Una volta installati, i plugin attivi possono essere verificati in qualsiasi momento all'interno di una sessione Claude Code con il comando slash: `/skills`

2.3 Gestione dei plugin nell'organizzazione

Accedendo all'interfaccia Cowork e cliccando su Customize, è possibile gestire sia le skill che i connettori MCP. I plugin hanno diversi stati di disponibilità a livello organizzativo:

- **Required:** installato obbligatoriamente per tutti gli utenti dell'organizzazione — non può essere rimosso.
- **Installed by default:** proposto a tutti ma rimovibile individualmente.
- **Available to install:** visibile e scaricabile volontariamente da chi ne ha bisogno.
- **Not available:** non mostrato agli utenti.

I plugin settati a livello organizzazione (ad es. per la firma di PDF, per query SQL, per project management) sono disponibili a tutti senza ulteriori installazioni.

2.4 Plugin fondamentali per la metodologia SDD

Di seguito i plugin più rilevanti introdotti nella lezione, con la descrizione del loro ruolo nel workflow:

Plugin	Scopo e utilizzo
SuperPowers	Il plugin principale per il Workflow SDD Avanzato. Implementa la tecnica Brainstorming & Specification con dialogo socratico guidato, attivabile con il comando /brainstorming.
Feature Dev (ufficiale Anthropic)	Workflow guidato per lo sviluppo di nuove funzionalità su progetti esistenti. Ideale quando l'architettura è già definita e si vuole aggiungere una feature in modo controllato.
CLAUDE.md Management	Skill per creare, migliorare e aggiornare il file CLAUDE.md. Include: revisione a fine sessione, miglioramento iterativo, analisi e suggerimenti sulla struttura esistente.
Code Simplifier	Analizza il codice e propone refactoring per ridurre la complessità. Utile nella fase di review post-implementazione.
Frontend Design	Strumenti per progettare interfacce utente di qualità, con suggerimenti su layout, componenti e stile.
Playwright	Permette a Claude di controllare il browser e automatizzare test funzionali o operazioni complesse.
Skill Creator	Skill per creare altre skill. Fondamentale per estendere le capacità di Claude con competenze personalizzate.
Playground	Genera pagine HTML interattive a partire da specifiche. Permette di visualizzare e modificare componenti UI in tempo reale.
MCP Server Dev	Strumento per lo sviluppo di nuovi server MCP personalizzati.

3 — Spec-Driven Development

La metodologia Spec-Driven Development (SDD) — sviluppo guidato dalle specifiche — è l'approccio metodologico al centro della lezione. Rappresenta uno dei pilastri dell'AI-assisted coding e definisce come collaborare in modo efficace con Claude per ottenere risultati prevedibili, riproducibili e di alta qualità.

3.1 Definizione

Lo Spec-Driven Development è una metodologia in cui l'AI non scrive codice liberamente, ma implementa una specifica dettagliata, approvata dall'utente, attraverso un ciclo controllato di pianificazione, esecuzione e revisione.

L'idea di base è semplice ma potente: separare il cosa fare (definito dalla specifica) dal come farlo (delegato all'AI). Prima si costruisce una specifica chiara e completa, poi si guida l'AI nell'implementarla in modo iterativo e verificato.

Il principio fondamentale

La qualità del risultato è direttamente proporzionale alla qualità della specifica. Ogni ora investita nella pianificazione ne risparmia tre o più di correzioni successive. Una specifica ben scritta porta la copertura AI dal 60% al 95%.

3.2 SDD vs Vibe Coding

Il termine vibe coding descrive l'approccio non strutturato che molti utenti adottano con l'AI: si fornisce una richiesta generica e si correggono iterativamente i risultati. Lo SDD si differenzia radicalmente da questo approccio:

Aspetto	Vibe Coding vs Spec-Driven Development
Approccio	Vibe: "Fai qualcosa" → correggi → correggi ancora. SDD: Specifica → Piano → Esecuzione controllata → Review.
Controllo	Vibe: l'AI decide cosa fare e come. SDD: l'utente approva ogni fase prima di procedere.
Risultato	Vibe: imprevedibile, variabile da una sessione all'altra. SDD: riproducibile, prevedibile.
Iterazioni correttive	Vibe: molte, spesso post-hoc. SDD: poche o nessuna, grazie alla qualità della specifica.
Scalabilità	Vibe: non scalabile su progetti complessi. SDD: scalabile, mantenibile nel tempo.
Documentazione	Vibe: assente o incompleta. SDD: la specifica è documentazione di progetto.

3.3 I 5 principi fondamentali dello SDD

Lo Spec-Driven Development si fonda su cinque principi essenziali:

Principio 1 — Separazione del cosa dal come

L'umano definisce cosa vuole ottenere dal punto di vista funzionale, lasciando all'AI la libertà di decidere come implementarlo — salvo vincoli tecnici specifici. Non si scrive codice riga per riga: si descrivono le specifiche funzionali e si lasciano aperti i dettagli implementativi.

Attenzione agli estremi: una specifica troppo restrittiva limita la creatività dell'AI; una troppo vaga lascia spazio ad ambiguità, che portano l'AI a prendere decisioni autonome non previste — il peggior scenario possibile.

Principio 2 — Granularità controllata

Lo sviluppo non è mai one-shot: viene suddiviso in più componenti o task atomici, ciascuno revisionato prima di procedere al successivo. La granularità permette di mantenere il controllo e di identificare i problemi precocemente.

Principio 3 — Review iterativa con cicli di feedback

Dopo ogni fase di implementazione (o anche a punti intermedi) viene effettuata una review. Il processo è ciclico: se la review identifica problemi, si torna alla fase appropriata — specifica, piano o esecuzione — e si procede con il raffinamento.

Principio 4 — Controllo umano sulle decisioni critiche

L'umano rimane al centro del processo (Human in the Loop). Le decisioni sull'architettura, sulle scelte tecnologiche, sui trade-off critici non vengono delegate all'AI: vengono prese dall'utente, che approva esplicitamente ogni gate prima di procedere.

Principio 5 — La specifica come documentazione di progetto

La specifica non è un documento usa e getta: è la fonte di verità del progetto. Deve essere mantenuta aggiornata in parallelo con il software. Ogni modifica al software dovrebbe essere preceduta da un aggiornamento della specifica.

Questo permette di: ricreare il progetto da zero, inserire nuovi sviluppatori, riutilizzare la logica in contesti diversi, sfruttare appieno le capacità di riproduzione dell'AI.

3.4 Il ciclo di sviluppo SDD

Il ciclo di sviluppo dello SDD è composto da quattro fasi principali, che si ripetono iterativamente:

1. **Scrittura delle specifiche:** si definisce cosa fare, con quale livello di dettaglio e quali vincoli. Questa è la fase più critica.
2. **Implementazione:** Claude esegue lo sviluppo seguendo la specifica. L'umano monitora e può intervenire in qualsiasi momento.
3. **Review:** si verifica la coerenza del risultato con la specifica. Può essere fatta dall'utente, da Claude stesso, o da entrambi in parallelo.
4. **Raffinamento:** si corregge l'implementazione se non è coerente con la specifica, oppure si aggiorna la specifica se si scoprono gap o nuove esigenze.

3.5 I due tipi di specifiche

Nell'ambito dello SDD esistono due tipologie di specifiche, che spesso coesistono nello stesso documento:

- **Specifiche funzionali:** descrivono cosa deve fare la soluzione, come si deve comportare l'applicazione. Non entrano nel dettaglio implementativo. Sono il tipo principale di specifica discusso e utilizzato nel corso.
- **Specifiche tecniche:** descrivono vincoli implementativi specifici (framework da usare, livelli di performance, tecnologie obbligatorie, regole architetturali). Si aggiungono alle funzionali quando esistono vincoli precisi.

3.6 I 7 componenti di una specifica efficace

Una specifica completa ed efficace dovrebbe contenere i seguenti sette elementi:

Componente	Contenuto e scopo
1. Overview	Contesto generale del problema. Descrive cos'è il progetto, perché esiste, per chi è pensato. Analogo alla descrizione di un progetto.
2. Requisiti funzionali	Il COSA fare: come deve comportarsi l'applicazione, quali funzionalità deve esporre, come deve rispondere agli input dell'utente.
3. Requisiti non funzionali	Vincoli trasversali: sicurezza, performance, accessibilità, scalabilità, disponibilità. Spesso trascurati ma critici.
4. Vincoli tecnici	Scelte tecnologiche obbligatorie: framework specifici, database imposti, pattern architetturali da seguire.
5. Criteri di accettazione	Come verificare che la specifica sia stata rispettata. Definisce il confine tra "fatto" e "non fatto".
6. Casi d'uso ed esempi	Scenari d'uso concreti che illustrano il comportamento atteso. Riducono le ambiguità interpretative.
7. Gestione degli errori	Come la soluzione deve reagire agli errori: messaggi da mostrare, comportamenti di fallback, strategie di retry.

4 — Workflow SDD Tradizionale (Planning Mode)

Il Workflow SDD Tradizionale è il punto di partenza storico della metodologia. Si basa sull'utilizzo del Planning Mode di Claude Code e di un prompt strutturato come specifica iniziale.

Nonostante sia stato superato dal Workflow Avanzato per i progetti complessi, rimane uno strumento valido per task ben definiti e di scope limitato.

4.1 Le 6 fasi del Workflow Tradizionale

Il workflow è articolato in sei fasi sequenziali, ognuna delle quali richiede approvazione esplicita prima di procedere:

Fase	Descrizione
1 — Specifica iniziale	Si scrive un prompt strutturato con obiettivo, requisiti, contesto e vincoli. Si richiede esplicitamente l'avvio del Planning Mode per impedire che Claude parta immediatamente con il codice.
2 — Q&A con Claude	Claude identifica le ambiguità nella specifica e fa domande per colmarle. Questa fase è cruciale: le decisioni prese qui entrano nella specifica raffinata e nel piano di sviluppo.
3 — Piano di sviluppo	Sulla base delle risposte al Q&A, Claude genera un piano di sviluppo dettagliato (salvato come file Markdown, es. development-plan.md). Include schema DB, componenti, metodi, dipendenze.
4 — Specifica raffinata	Il prompt iniziale viene aggiornato incorporando le decisioni del Q&A. Questa specifica migliorata diventa l'input per la fase di esecuzione.
5 — Esecuzione controllata	Claude implementa il piano diff-by-diff, richiedendo approvazione per ogni modifica. L'utente rivede ogni cambiamento prima che venga applicato.
6 — Review e validazione	Verifica post-implementazione: funzionalità rotte, feature mancanti, possibili miglioramenti. Può essere fatta dall'utente e/o da Claude stesso.

4.2 Caso pratico: EV-002 — Configurable Working Hours

Per illustrare il Workflow Tradizionale, la lezione ha presentato un caso reale di evolutiva sviluppata su Task Management, una applicazione gestionale interna di Gamma. La richiesta era di rendere configurabile il numero di ore lavorative giornaliere per ogni risorsa (default 8h, con possibilità di definire un calendario settimanale e gestire eccezioni con granularità di 30 minuti) e di integrare questa logica nel calcolo automatico del diagramma di Gantt.

Il prompt iniziale includeva obiettivo chiaro e requisiti principali, ma era troppo vago su aspetti critici come il modello gerarchico a tre livelli (default > weekly pattern > override), la migrazione della logica delle assenze, l'integrazione delle festività e le specifiche UI. Questo ha richiesto molti cicli di review post-implementazione.

Lezione appresa

Investire più tempo nella specifica iniziale è sempre conveniente. Claude ha identificato 5+ ambiguità durante il Q&A — ognuna di esse avrebbe generato bug e correzioni costose se non fosse stata chiarita prima dell'implementazione.

Come misura dell'impatto della specifica, alla fine dello sviluppo è stato chiesto a Claude di valutare il prompt iniziale e di riscrivere la specifica ideale che avrebbe portato al risultato in modo quasi one-shot. Il confronto è stato illuminante:

Aspetto	Prompt originale vs Specifica ottimizzata
Lunghezza	~35 righe / ~150 righe
Requisiti funzionali	3 generici / 7 dettagliati (FR-1...FR-7)
Schema dati	Assente / 4 entità con campi e vincoli
Specifiche UI	Assenti / labels, editor, layout, messaggi
Business rules	Implicite / esplicite (festività, cascade, ...)
Copertura AI stimata	60-70% / ~95%
Iterazioni correttive	Molte / poche o nessuna

5 — Workflow SDD Avanzato (Brainstorming & Specification)

Il Workflow SDD Avanzato è l'evoluzione moderna della metodologia, introdotta da poche settimane al momento della lezione grazie al plugin SuperPowers. Trasforma radicalmente la fase di pianificazione, sostituendo il Planning Mode con una sessione di brainstorming guidata che produce specifiche di qualità nettamente superiore.

5.1 L'idea di fondo

Il problema principale del Workflow Tradizionale è che la qualità della specifica dipende interamente dall'utente, che deve anticipare tutti i requisiti, le ambiguità, i vincoli e i casi limite prima di iniziare. In pratica, questo è quasi impossibile per problemi complessi.

Il Workflow Avanzato sposta parzialmente questa responsabilità: attraverso un dialogo socratico guidato (brainstorming), l'AI collabora attivamente con l'utente per esplorare il problema, identificare i gap, prendere decisioni architetturali e costruire insieme una specifica completa.

Il risultato

Una specifica prodotta con il Workflow Avanzato è mediamente molto più completa di una scritta manualmente, richiede meno tempo all'utente e riduce drasticamente le iterazioni correttive post-implementazione.

5.2 Le fasi del Workflow Avanzato

Il workflow si divide in due macrofasi — brainstorming ed esecuzione — articolate in cinque passi:

FASE DI BRAINSTORMING (preferibilmente su Claude Code, in alternativa su Desktop)

1. Prompt originale: si fornisce la descrizione del problema e l'obiettivo iniziale. Non deve essere perfetto — serve come punto di partenza per la sessione.
2. Q&A esplorativo: Claude analizza il prompt, esplora la codebase (se presente), propone opzioni architetturali e pone domande mirate per chiarire ambiguità, scope, decisioni tecniche.
3. Specifica dettagliata: al termine del dialogo, Claude genera automaticamente un documento Markdown completo con requisiti funzionali, edge cases, schema dati, decisioni architetturali. L'utente la revisiona e approva.
4. Piano di implementazione: dalla specifica approvata, Claude genera un piano di sviluppo dettagliato file-by-file, metodo-by-metodo, con dipendenze e ordine di esecuzione.

FASE DI ESECUZIONE (su Claude Code)

5. Esecuzione controllata (e review): Claude implementa il piano, con possibilità di parallelizzare i task tramite sub-agents. La review avviene in modo integrato.

5.3 Il plugin SuperPowers e il comando `/brainstorming`

Il cuore del Workflow Avanzato è il plugin SuperPowers, sviluppato da Jess Vincent.

Si attiva con il comando:

`/brainstorming`

Seguito dal proprio prompt iniziale. Claude riconosce il comando, carica il plugin SuperPowers e avvia la sessione di dialogo socratico.

Caratteristiche della sessione:

- Claude legge il `CLAUDE.md` del progetto (se presente) per avere il contesto di base.
- Esplora la codebase esistente per capire l'architettura corrente.
- Pone domande interattive — alcune testuali, alcune con menu di scelta cliccabili.
- Propone architetture, confronta opzioni, raccomanda scelte motivate.
- Può avviare un server locale e mostrare mockup interattivi nel browser per discutere il layout e la navigazione.
- Alla fine genera automaticamente la specifica (file `.md`) e la sottopone a self-review prima di presentarla all'utente.

5.4 Caso pratico: EV-003 — Modified Gantt Date Calculation

La seconda evolutiva dimostrata riguarda la modifica del calcolo delle date nel Gantt di Task Management. Il problema: il sistema assumeva che tutte le risorse lavorassero 8 ore al giorno. Con la nuova logica di orari lavorativi configurabili (EV-002), un task da 16 ore non doveva più essere calcolato come 2 giorni per tutti, ma in base all'effettiva disponibilità oraria di ciascuna risorsa.

L'intera evolutiva — brainstorming, specifica, pianificazione, sviluppo, test, refactoring, manuale utente e rilascio — è stata completata in circa una giornata lavorativa con il Workflow Avanzato. Breakdown temporale approssimativo:

- ~3 ore: brainstorming e pianificazione (inclusa esplorazione del codice)
- ~2 ore: sviluppo
- ~1 ora: test e bug fixing
- ~4 ore: code refactoring
- ~30 minuti: redazione manuale utente, rilascio e condivisione

Anche con brainstorming approfondito possono esistere gap

Durante la review è emerso un vincolo tecnico non previsto: la separazione MVC-WebAPI richiedeva un Controller HTTP + batch model che la specifica non specificava. Questo ha richiesto un ciclo di correzione aggiuntivo. Il messaggio: il brainstorming riduce drasticamente i gap, ma non li elimina al 100%. La review post-implementazione rimane necessaria.

5.5 Confronto tra Workflow Tradizionale e Avanzato

Aspetto	Planning Mode vs Brainstorming & Specification
Fase iniziale	Prompt strutturato diretto / Brainstorming esplorativo con Q&A approfondito
Specifica	Evolve dal prompt dopo Q&A / Documento formale completo con FR, edge cases, schema
Piano di sviluppo	Generato in planning mode / Writing-plans: file-by-file, metodo-by-metodo
Esecuzione	Diff-by-diff controllata / Subagent-driven con possibile parallelizzazione
Qualità specifica	Dipende dall'utente / Co-prodotta con l'AI
Tempo di planning	Minore / Maggiore (ma ripaga ampiamente)
Iterazioni correttive	Mediamente molte / Poche o nessuna

5.6 Quando usare quale workflow

Usa il Workflow Tradizionale (Planning Mode) quando...
Il problema è ben definito e chiaro fin dall'inizio
L'architettura è già nota e consolidata
Si tratta di feature standard (CRUD, UI, report)
I requisiti sono chiari e completi
Lo scope è limitato (pochi file coinvolti)
C'è urgenza e si privilegia la velocità di delivery

Usa il Workflow Avanzato (Brainstorming & Specification) quando...
Il problema richiede esplorazione e scoperta
L'architettura deve essere progettata o rivista
Si tratta di algoritmi complessi o refactoring cross-layer
Ci sono integrazioni tra layer e servizi
Lo scope è ampio (molti file, dipendenze multiple)
Si vuole sviluppare un prototipo da zero
La qualità del risultato è più importante della velocità iniziale

6 — Dimostrazione pratica: App Shopping List

La parte finale della lezione ha incluso una dimostrazione live del Workflow SDD Avanzato applicato allo sviluppo di un prototipo di app — una lista della spesa — partendo da zero.

6.1 Avvio del brainstorming

La sessione è stata avviata da Claude Code (interfaccia desktop) con il comando `/brainstorming` seguito dal prompt iniziale. Il `CLAUDE.md` del progetto era già stato preparato in anticipo con le informazioni di base sul progetto.

Claude ha immediatamente:

- Caricato il `CLAUDE.md` in memoria
- Avviato la sessione di dialogo socratico
- Proposto di mostrare mockup interattivi nel browser (funzionalità nuova, presa d'iniziativa da Claude)

6.2 Sessione di Q&A esplorativo

Le domande poste da Claude durante il brainstorming hanno coperto:

- Scope: MVP o applicazione completa? (scelta: MVP, alcune feature rimandate alla fase 2)
- Backend: online o offline-first? (scelta: solo offline, niente backend)
- Autenticazione: guest mode o utenti registrati? (scelta: guest mode)
- Salvataggio: modal di conferma o autosalvataggio silenzioso? (scelta: autosalvataggio)
- Architettura: vanilla JS, Lit o Preact? State management pattern?

Per le domande tecniche (scelta del framework, pattern di state management), è stato suggerito di aprire un'altra sessione Claude per capire il significato delle opzioni proposte, oppure di procedere con la raccomandazione di Claude. Questo è il modo corretto di gestire le domande tecniche per chi non ha background di sviluppo.

6.3 Generazione della specifica e del piano

Al termine del brainstorming, Claude ha:

1. Generato automaticamente il documento di specifica
2. Effettuato una self-review della specifica
3. Presentato la specifica all'utente per approvazione
4. Su approvazione, generato il piano di implementazione dettagliato

6.4 Risultato del prototipo

Il prototipo generato in un'unica sessione one-shot (senza controllo umano durante l'implementazione, come dimostrazione delle capacità del sistema) includeva:

- Creazione e gestione di multiple liste della spesa con colori personalizzati
- Aggiunta, categorizzazione e ordinamento degli articoli
- Unità di misura per gli articoli
- Persistenza locale (IndexedDB)
- Autocomplete nella ricerca articoli
- Interfaccia responsive con menu di navigazione

Pur presentando alcuni bug, il risultato dimostra la potenza dell'approccio: un prototipo funzionale completo in una singola sessione guidata.

7 — Esercizi assegnati

Al termine della lezione sono stati assegnati quattro esercizi progressivi da completare prima della sessione successiva. Gli esercizi si basano sullo sviluppo di una web app per la gestione delle attività personali (to-do list) e coprono l'intero ciclo dello SDD.

Esercizio 1 — Scrittura manuale delle specifiche

Scrivere manualmente le specifiche funzionali (ed eventualmente tecniche) per una web app dedicata alla pianificazione e organizzazione delle proprie attività. L'app deve essere progettata come la si vorrebbe realmente usare.

Obiettivo: sviluppare la capacità di tradurre un'idea in requisiti concreti, prima ancora di coinvolgere l'AI.

Esercizio 2 — Configurazione dell'ambiente Claude Code

Generare la configurazione dell'ambiente di sviluppo Claude Code per il progetto dell'esercizio 1. Creare il file `CLAUDE.md` di base (workflow Lite) con le informazioni essenziali sul progetto.

Obiettivo: familiarizzare con la struttura del `CLAUDE.md` e con il processo di configurazione iniziale.

Esercizio 3 — Scrittura delle specifiche con Brainstorming

Partendo dalla propria idea, utilizzare il Workflow SDD Avanzato (/brainstorming) per produrre una specifica di qualità attraverso il dialogo socratico con Claude. Rispondere alle domande di Claude in modo dettagliato e approvare la specifica finale.

Obiettivo: sperimentare il Workflow Avanzato e valutare la differenza qualitativa rispetto alla specifica scritta manualmente nell'Esercizio 1.

Esercizio 4 — Implementazione del prototipo

Sulla base della specifica e del piano di sviluppo prodotti nell'Esercizio 3, implementare il prototipo della web app utilizzando le tecniche SDD. Partire dall'approvazione del piano di sviluppo e procedere con l'esecuzione controllata.

Obiettivo: completare il ciclo end-to-end dello SDD, dall'idea al prototipo funzionante.

Nota sul percorso

I quattro esercizi sono sequenziali. Se si incontra difficoltà in uno dei passaggi, non bloccarsi: documentare il punto di blocco e portarlo alla lezione successiva, dove verrà affrontato e chiarito insieme.

Riepilogo e Concetti Chiave

Di seguito i concetti fondamentali della quarta lezione, organizzati per tema.

Plugin di Claude e Claude Code

- I plugin estendono le capacità di Claude con skills, connettori MCP, sub-agents e hooks.
- I plugin di Claude Code si installano da riga di comando
- Richiedono il riavvio di Claude Desktop dopo l'installazione.
- Il plugin SuperPowers è il fondamento del Workflow SDD Avanzato.

Metodologia Spec-Driven Development

- Lo SDD separa il COSA (specifica) dal COME (implementazione AI).
- Il controllo umano è centrale: ogni fase richiede approvazione prima di procedere.
- La qualità della specifica determina direttamente la qualità del risultato.
- Ogni ora investita nella specifica risparmia ore di correzioni post-implementazione.

Workflow Tradizionale vs Avanzato

- Tradizionale (Planning Mode): prompt strutturato → Q&A → piano → esecuzione. Veloce, ideale per task ben definiti.
- Avanzato (Brainstorming & Spec): dialogo socratico → specifica dettagliata → piano → esecuzione con sub-agents. Più lento inizialmente, qualità superiore, ideale per problemi complessi.